COMP9414 Assignment 1
Search and planning for an equipment service robot
Term 3, 2026

UNSW Sydney

Dates	Release and deadline: see Moodle.
Worth	20 marks
Submission	Electronic submission via Moodle

1 Introduction

An equipment service robot has two kinds of decisions to make. It must choose a route through a workshop, where the fewest movements need not give the lowest travel cost. It must also service and test equipment using a shared workbench and a tool that needs resetting between uses. Reaching the right place does not, by itself, complete a service job.

You will formulate search problems, trace and implement search algorithms, investigate the effect of a heuristic, and build a forward planner. The assignment covers the search and planning material from weeks 1 and 2. The route-finding and equipment-servicing tasks are separate models: you do not need to combine them into one controller.

2 Background

2.1 Agents and search

A search problem specifies an initial state, available actions, a transition model, a goal test, and a path-cost function. Its state space contains the possible states; a search tree represents the paths considered while solving the problem. Different paths can reach the same state.

The frontier contains entries still waiting to be examined. Depth-first search (DFS) uses a stack, breadth-first search (BFS) uses a queue, and uniform-cost search (UCS) uses cumulative path cost as priority. A* uses $f(n) = g(n) + h(n)$, combining the cost so far with an estimate of the remaining cost. You will implement these four algorithms and explain their observed behaviour. No proof of A* optimality or heuristic admissibility is required.

2.2 Planning

In STRIPS planning, a state is a set of facts. A grounded action names specific objects and has preconditions, add effects, and delete effects. An action a is applicable in state s when $\mathrm{Pre}(a) \subseteq s$. Its result is

$$\mathrm{Result}(s, a) = (s - \mathrm{Del}(a)) \cup \mathrm{Add}(a).$$

Facts not mentioned in the effects remain unchanged. A state satisfies the goal when it contains every goal fact. Forward planning searches over these fact sets. In this assignment, every planning action costs one, so BFS searches for a plan with the fewest actions.

3 Assignment specification

3.1 Supplied problems

Download `assignment1_data_t3.zip` from Moodle and extract it. Place its `data/` folder next to your notebook. Load the JSON files for your automated runs; the tables in this document support your written answers.

Table 1: Files supplied for Term 3.

File	Use
`data/graphs/workshop.json`	Small workshop map for hand traces and search comparisons.
`data/graphs/routes.json`	Larger map for search and heuristic comparisons.
`data/graphs/detour.json`	Workshop map with the ramp–lift connection closed.
`data/planning/service.json`	Two units need servicing and testing.
`data/planning/ready.json`	The camera is already serviced, but still needs testing.
`data/planning/offline.json`	The tool reset action is unavailable.
`data/manifest.json`	How the files were created, including the random seed used for the larger graph.

These examples let you test and explain your algorithms. During your discussion, your tutor may ask you to run or trace a small modified problem using the same schema. Your solvers must use the supplied states, goals, edges, and actions; do not hard-code solutions or recognise individual filenames.

3.1.1 Graph problem schema

Each graph is undirected. Every edge has a positive cost and joins two distinct nodes. Node identifiers are unique, and there is at most one edge between a pair of nodes. The start and goal are node identifiers.

```
{
  "nodes": [{"id": "depot", "x": 0.0, "y": 0.0}, ...],
  "edges": [{"u": "depot", "v": "aisle", "cost": 2.0}, ...],
  "start": "depot",
  "goal": "test_bay"
}
```

Coordinates and costs use the same distance units. In the supplied graphs, each edge costs at least the straight-line distance between its endpoints. Coordinates guide A*; they do not determine which edges exist. Ignore extra fields such as `id`, `seed`, or `metadata`.

3.1.2 Planning problem schema

Each planning file lists the initial facts, goal facts, and all grounded actions. Facts are strings, including their punctuation. You can treat each string as a single symbol: you do not need a parser or a grounding procedure. For example, one action record is:

```
{
  "name": "mount(camera)",
  "preconditions": ["on_rack(camera)", "bench_free"],
  "add": ["on_bench(camera)"],
  "delete": ["on_rack(camera)", "bench_free"]
}
```

The outer problem has the following structure:

```
{
  "initial": ["bench_free", "tool_ready", ...],
  "goal": ["tested(camera)", "on_rack(camera)", ...],
  "actions": [{"name": "mount(camera)", ...}, ...]
}
```

Action names are unique. All preconditions and goal conditions are positive facts. A fact absent from the current state is false in this model. Each action's add and delete lists are disjoint. Ignore any additional metadata. Your planner must read the action records rather than implement special cases for `mount`, `service`, or particular equipment names.

3.2 Stable solver interfaces

Define these functions in executable notebook cells before your evaluation cell:

```
def solve_graph(problem: dict, algorithm: str) -> dict:
    ...

def solve_planning(problem: dict) -> dict:
    ...
```

The graph algorithm argument accepts `dfs`, `bfs`, `ucs`, and `astar`. The planning function implements forward BFS. Both functions return the fields in Table 2. Do not modify the input problem in either function.

Table 2: Required solver return fields.

Field	Meaning
`status`	`found`, `not_found`, or `error`. An unreachable goal is `not_found`.
`path`	Graph node identifiers from start to goal, or an empty list if no path is found.
`plan`	Grounded action names, or an empty list if no plan is found.
`cost`	Graph path cost or planning action count; `None` if no solution is found.
`expanded_order`	Expanded graph node identifiers, or planning states as sorted lists of fact strings.
`expanded_count`	Length of `expanded_order`.
`generated_count`	Candidate successors produced; see Section 8.
`frontier_peak`	Largest number of entries held in the frontier.
`message`	Optional explanation for an `error` result.

Graph results include `path`; planning results include `plan`.

3.2.1 Ordering and repeated states

Use the following conventions for both your code and your hand traces.

- **DFS and BFS**: order neighbours alphabetically by node identifier. Mark a node discovered when you add it to the frontier, including the start. Add each node only once. BFS appends eligible neighbours in alphabetical order. DFS pushes them in reverse alphabetical order so the alphabetically first eligible neighbour is popped next.
- **UCS and A***: order frontier entries first by priority (g or $g + h$), then lexicographically by the complete path tuple, then by insertion order. Generate neighbours alphabetically. Keep the best pair (g, path tuple) found for each node. Add a new entry only when this pair improves: a cheaper path wins, or, at equal cost, the lexicographically smaller path wins.
- Keep older entries in the priority queue after inserting a better path. When popped, skip an entry if its cost and path no longer match the recorded best pair. Such an entry is *stale*; it is not expanded. A later improved path may be added even if that node was expanded earlier.
- **Planning BFS**: consider applicable grounded actions in alphabetical order of their complete name strings. Mark each fact set discovered when you add it to the queue. Equal fact sets are the same state, regardless of the order in which their facts are stored.

Test the goal when a current entry is popped, after discarding any stale entry. Include a popped goal in the expansion order, but do not generate its successors. If the initial state already satisfies the goal, return cost zero and either the one-node path or the empty plan. If the frontier becomes empty without a goal, return `not_found`.

Use the stored numeric values during search; round only when displaying results. Standard Python tuple comparison is sufficient for the ordering rules above; see Appendix B for a short Python reference. We assess valid inputs using these schemas; extensive input validation is not required.

3.3 Solution format

Submit your complete solution as `zID_assignment1.ipynb`, replacing `zID` with your student identifier. Use Markdown cells for written answers and hand traces, and code cells for implementations, evaluation, tables, and plots. Keep your answers in the order of Parts A–D.

Your notebook must define both solver functions, display executed results for the required runs, and include the reusable evaluation cell described in Section 9.1. Separate Python solver files or a separate written report are not required. Restart the kernel and run all cells before submission so that the notebook works without variables left over from an earlier session.

4 Part A: problem formulation and search traces

4.1 Workshop routes

Use `data/graphs/workshop.json`, whose edges are shown in Table 3. The robot starts at `depot` and must reach `test_bay`. An edge can be traversed in either direction for the stated cost.

Table 3: Workshop graph. Start: `depot`. Goal: `test_bay`.

From	To	Cost
depot	aisle	2
depot	stores	2
aisle	ramp	3
aisle	workshop	6
stores	ramp	1
ramp	lift	2
workshop	lift	2
workshop	test_bay	3
lift	test_bay	4

1. Define the states, initial state, actions, transition model, goal test, and path cost for this route-finding problem.

2. Trace DFS, BFS, and UCS by hand using Section 3.2.1. Show the initial frontier and the frontier after each of the first three expansions. Write entries as paths with cumulative costs, with the next entry to be removed first. Label any stale UCS entries that remain queued.

3. Give the complete expansion order, returned path, number of edges in that path, and total cost for each algorithm. Explain any differences between the returned paths, including the effect of edge costs on BFS and UCS.

4.2 Representing repeated states

Show two different paths from `depot` to `ramp`. Explain why these are different search-tree entries but reach the same state in this model. Then explain why UCS must allow a cheaper path to replace an earlier path to that state. Use an example from your trace.

Your hand traces are part of the written answer. You may use your later implementation to check them, but include the frontier contents and reasoning.

5 Part B: implement graph search

5.1 Algorithms

Implement `solve_graph(problem, algorithm)` in your notebook. Use a stack for DFS, a queue for BFS, and a priority queue for UCS and A*. Use $g(n)$ as the UCS priority and $g(n) + h(n)$ as the A* priority, with the straight-line heuristic defined in Part C.

Follow the ordering and repeated-state rules in Section 3.2.1. Your code must terminate on finite graphs with cycles, report unreachable goals, and reconstruct a path when it finds a goal. Implement the search loops yourself. You may use standard-library tools such as `collections.deque` and `heapq`, but you may not call a library function that performs the required search for you.

5.2 Comparison and route closure

Run all four algorithms on `workshop.json`, `routes.json`, and `detour.json`. For each run, report the status, path, cost, expansion order, expanded count, generated count, and frontier peak.

Present a comparison table and at least one figure with separate panels or axes for returned cost and expanded count. Keep the distance and node-count units distinct. Long paths and expansion orders may appear below the table.

Compare the results for `workshop.json` and `detour.json`. The only change is removal of the undirected ramp–lift edge. Explain how this closure affects the cheapest route and the search effort. Use the results to explain why fewer edges in a returned path need not mean a lower path cost.

5.3 Small checks

Make two independent copies of `workshop.json` in your notebook:

1. Set the goal equal to the start.
2. Keep the original start and goal, but remove every edge incident to `test_bay`, leaving that node in the node list.

Run all four algorithms on each copy. Check the status, path, cost, and expansion metrics against the interface rules. Include these results in your evaluation output. Keep the supplied files unchanged.

6 Part C: A* heuristic behaviour

6.1 Straight-line distance

Use the same heuristic for every A* run:

$$h(n) = \sqrt{(x_n - x_g)^2 + (y_n - y_g)^2},$$

where the subscript g denotes the goal node. UCS does not use the coordinates. The heuristic estimates the distance still to travel; it does not replace $g(n)$, the actual path cost so far.

6.2 Comparing frontier choices

Use the UCS and A* results for both `workshop.json` and `routes.json`. You may reuse the runs from Part B.

1. Compare returned cost, expanded count, generated count, and frontier peak on each graph.
2. For each graph, identify the first point where UCS and A* expand different nodes. Show the competing path entries and their g, h, and $g+h$ values immediately before this choice.
3. Explain how the heuristic changes the work done. Refer to your traces and metrics; do not assume that every metric must improve together.

6.3 Removing coordinate information

Make a copy of `routes.json` and set every node's coordinates to $(0, 0)$. Keep the edges, costs, start, and goal unchanged. Run A* on this copy and include its full returned fields in your evaluation output.

Compare this run with UCS and the original A* run. Explain the resulting heuristic values and frontier priorities, and whether the returned path, expansion order, and metrics match UCS under the specified tie-breaking. Finally, describe one reason why straight-line distance can give little guidance when available routes force a detour. Base your answers on the algorithm and observed results; no formal heuristic proof is required.

7 Part D: STRIPS modelling and forward planning

7.1 Equipment service domain

Two units, `camera` and `sensor`, share one workbench and one service tool. The robot handles one unit on the bench at a time. Servicing a unit uses the tool; resetting the tool requires an empty bench. All actions are deterministic and occur one at a time.

Each unit `u` is on the rack or bench, needs service or is serviced, and is untested or tested. The corresponding facts are `on_rack(u)`, `on_bench(u)`, `needs_service(u)`, `serviced(u)`, `untested(u)`, and `tested(u)`. Shared facts describe the bench and tool: `bench_free`, `tool_ready`, and `tool_used`.

In `service.json`, each unit initially has `on_rack(u)`, `needs_service(u)`, and `untested(u)`. The only other initial facts are `bench_free` and `tool_ready`.

The goal requires `tested(camera)`, `tested(sensor)`, `on_rack(camera)`, `on_rack(sensor)`, and `bench_free` simultaneously. It does not require the tool to finish ready.

The action meanings are as follows. Facts not mentioned as changing remain unchanged.

- `mount(u)` requires the unit on the rack and a free bench. It places the unit on the bench, removing its rack fact and `bench_free`.
- `service(u)` requires the unit on the bench, needing service, and a ready tool. It makes the unit serviced and the tool used, removing `needs_service(u)` and `tool_ready`.
- `test(u)` requires the unit on the bench, serviced, and untested. It makes the unit tested and removes `untested(u)`.
- `store(u)` requires the unit on the bench and tested. It returns the unit to the rack and frees the bench, removing `on_bench(u)`.
- `reset_tool()` requires a free bench and a used tool. It makes the tool ready and removes `tool_used`.

The supplied JSON contains the grounded actions for these rules. Read it alongside this description. Only the listed actions can change a state.

7.2 Action representation and plan diagnosis

Give the precondition, add, and delete sets for `service(camera)` and `reset_tool()`. Then consider this proposed plan for `service.json`:

```
1. mount(camera)
2. service(camera)
3. test(camera)
4. reset_tool()
5. store(camera)
6. mount(sensor)
7. service(sensor)
8. test(sensor)
9. store(sensor)
```

1. Identify the first action that is not applicable. Show the complete fact set just before it and name every missing precondition.
2. Explain why omitting that action also prevents this sequence from completing the goal. Identify where the shortened sequence would fail.
3. Repair the plan. Give the full action sequence and explain which facts in its final state satisfy the goal.

7.3 Forward BFS planner

Implement `solve_planning(problem)` in your notebook. Read the initial facts, goal facts, and action records from the input dictionary. Your code must:

- maintain a queue of states and their plan prefixes;
- check positive preconditions using set inclusion;
- apply the delete and add effects using the STRIPS transition rule;
- test whether all goal facts hold together;
- avoid adding the same fact set to the queue more than once;
- return all required fields with deterministic action ordering.

You may represent a state using `frozenset(facts)`. A `frozenset` is a set whose contents cannot be changed. Unlike a normal `set`, it can itself be stored in another set or used as a dictionary key, which lets you record which states you have already visited. Convert each expanded state to a sorted list of facts when creating `expanded_order`, so your result can be written directly to JSON. The supplied actions are already grounded; the planner needs only set operations and a BFS loop. Implement this loop yourself rather than calling a complete planning solver.

7.4 Planning variants and a modelling fault

Run your planner on `service.json`, `ready.json`, and `offline.json`. The second case changes only the camera's initial service status: it starts serviced, on the rack, and untested. The third case has the baseline initial state but removes `reset_tool()` from the available actions. All three have the same goal.

Present the status, plan, plan length or `None`, expanded count, generated count, and frontier peak in a comparison table. Include expansion orders in your saved results. Explain any change in plan length and search effort. If a goal is unreachable, identify the fact that would be needed and why the available actions cannot restore it at the required time.

Finally, suppose `mount(u)` no longer deletes `bench_free`. Give a short action sequence that the faulty model would allow but the one-unit bench should prevent. Identify the missing delete effect and explain why changing BFS tie-breaking would not repair the model. You do not need to run a separate planner experiment for this written question.

8 Evaluation metrics

Use these counting conventions in every run:

- **Returned cost**: sum of graph edge costs, or number of planning actions. Use `None` for an unreachable goal.
- **Expanded count**: number of current entries popped for a goal test and possible successor generation. Include the initial state and any popped goal; exclude skipped stale priority-queue entries.
- **Generated count**: count each graph neighbour considered from an expanded non-goal node before duplicate or cost filtering. For planning, count one successor for each applicable action, before checking whether the resulting state has been seen. Count repeated candidates, but not inapplicable actions. Do not count the initial state as generated.

- **Frontier peak**: maximum number of stored frontier entries, including any stale entries still in a priority queue. Initialise this count to one for the initial entry. Update it after additions.
- **Expansion order**: record entries when counted as expanded, using the representations in Table 2.

For an already-satisfied initial goal, the expanded count is one, generated count is zero, and frontier peak is one. When comparing normal runs, connect cost to search effort: a short expansion trace alone does not establish that the returned path has low cost. You do not need to measure wall-clock time.

9 Assessment and evaluation

9.1 Evaluation process

Your tutor will assess your submission during an individual discussion session. See Moodle and your tutor's announcement for the dates and booking arrangements. You must attend your arranged discussion. If you do not discuss your submission, you will receive zero marks for the assignment.

Your notebook must include a reusable evaluation cell that:

1. loads the graph and planning JSON files using the assignment schemas;
2. calls the `solve_graph` and `solve_planning` functions defined earlier in the notebook;
3. runs all four graph algorithms on the three supplied maps and the two Part B checks, runs the Part C zero-coordinate A* comparison, and runs the planner on all three planning cases;
4. prints readable summary tables and displays the returned paths or plans and expansion orders;
5. writes the full returned fields to `zID_results.json`, using distinct case and algorithm labels.

You may define small evaluation helpers in preceding cells. Keep filenames in one clearly identified list or mapping so your tutor can change an input file without editing your solver. Label the derived graph copies `at_goal`, `isolated_goal`, and `zero_coordinates` in the results. UCS and A* runs already required in Part B can be reused in Part C. Appendix A shows a loading example and result-file structure.

Run the evaluation cell before submission. During discussion, your tutor may ask you to rerun it on a small check instance or trace a modified problem by hand. If you do not include the evaluation cell, you will lose 2 marks from your total assignment mark.

9.2 Assessment breakdown

Table 4 gives the marks for the notebook and discussion.

Your tutor will choose one topic from each discussion group:

- **Foundations**: problem formulation; state space versus search tree; repeated states; frontier ordering and metric definitions.
- **Search behaviour**: DFS and BFS; UCS path replacement; the route-closure results; A* frontier choices and the zero-coordinate run.

Table 4: Assignment assessment breakdown.

Assessment component	Marks
Assignment implementation and notebook	
Part A: Problem formulation and hand search traces	1.5
Part B: Graph search implementation and comparison	2.0
Part C: Heuristic behaviour analysis	1.5
Part D: STRIPS modelling and forward BFS planning	3.0
Understanding and discussion	
Group 1: Foundations	4.0
Group 2: Search behaviour	4.0
Group 3: Planning models	4.0
Total	20.0

- **Planning models**: preconditions and effects; shared bench and tool facts; plan diagnosis; forward BFS; unreachable goals; modelling faults.

Be ready to explain your code and connect your answers to the states, frontiers, and results in your notebook. You are assessed on your reasoning and implementation, not on memorising particular output sequences.

10 Submission requirements

Submit one ZIP archive named `zID_assignment1.zip` through Moodle. It must contain:

1. `zID_assignment1.ipynb`, with executed outputs;
2. `zID_results.json`, written by the reusable evaluation cell;
3. any generated plots, screenshots, or small helper files needed by your notebook. Plots embedded in the notebook do not need separate copies.

Your notebook must include the Part A formulation and traces, Part B implementations and comparisons, Part C heuristic experiment, Part D action sets, diagnosis, planner, comparison and modelling-fault answer, and the reusable evaluation cell. The supplied `data/` folder will be placed beside the notebook for marking; you do not need to submit another copy.

Use Python 3.11 or later. The solver and JSON-loading code must run using the Python standard library. You may use Matplotlib for plots. The `ai9414` demos may help you study search behaviour, but the package is not required to run this assignment. Its office-delivery planning helpers use a different schema from the grounded action files supplied here. All submitted search and planning loops must be your own code.

11 Deadline and questions

The submission deadline is listed on the assignment page on Moodle.

Late submissions: a penalty of 5% of the maximum mark per day applies. If you submit more than 5 days late, you will receive a mark of zero.

Use the Moodle forum for assignment questions. We prioritise questions asked there. Do not share code or complete answers publicly. If necessary, contact course staff by email. For discussion-session questions, contact your tutor.

12 Academic integrity

Complete this assignment individually. You may discuss concepts and high-level approaches with other students, consult course materials and course staff, and use AI tools for learning concepts or debugging. You must not share code, copy code, submit code written by someone else, or post assignment content or solutions on external forums.

You are responsible for understanding every part of your submission. During the discussion, your tutor may ask you to explain a design choice, trace a frontier update, modify a small case, or diagnose why a planning action is not applicable.

A Loading data and recording results

Python's standard library can load both input schemas:

```
import json
from pathlib import Path

data_dir = Path("data")
with (data_dir / "graphs" / "workshop.json").open() as stream:
    graph = json.load(stream)
with (data_dir / "planning" / "service.json").open() as stream:
    planning_problem = json.load(stream)

graph_result = solve_graph(graph, "ucs")
planning_result = solve_planning(planning_problem)
```

Use a separate record for each case and algorithm. For example:

```
results = {
    "graphs": {"workshop": {"ucs": graph_result}},
    "planning": {"service": planning_result},
}
with Path("zID_results.json").open("w") as stream:
    json.dump(results, stream, indent=2)
```

Extend this structure to include every required run. Replace `zID` in the output filename with your student identifier. JSON stores Python `None` as `null`; use lists rather than sets in returned results.

For the graph modifications, use `copy.deepcopy` to make independent copies, including the lists inside each graph dictionary. Simply assigning the graph to another variable does not copy it. For planning, convert fact lists to sets and sort actions by `name`. Use the set operations from the background section to check applicability and compute successor states.

B Python quick reference

- **Stack (last in, first out).** Start with `stack = []`. `stack.append(item)` adds an item; `stack.pop()` removes and returns the most recently added item.
- **Queue (first in, first out).** Use `from collections import deque` and `queue = deque()`. `queue.append(item)` adds at the back; `queue.popleft()` removes and returns the oldest item. Check that a stack or queue is non-empty before removing an item, for example with `while queue:`.
- **Sets.** `set(facts)` keeps unique facts without an order; `set()` creates an empty set. Use `fact in state` for membership, `pre <= state` to check that all preconditions hold, `state - delete` to remove facts, and `state | add` to include facts. `sorted(state)` produces a sorted list for output.
- **Tuples.** A tuple is an ordered sequence that cannot be changed; `tuple(path)` converts a path list to one. Python compares tuples from left to right, using the first unequal item to decide their order. A shorter tuple comes first if it is a prefix of the other.
- **Priority queue.** After `import heapq`, an empty list `q = []` can serve as a priority queue. `heapq.heappush(q, entry)` adds an entry; `heapq.heappop(q)` removes the smallest one. Tuple entries follow the comparison rule above.

C References

1. Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson.
2. COMP9414 week 1 lecture: Agents and search.
3. COMP9414 week 2 lecture: Planning.
4. The `ai9414` Python package and course search demos.
5. Optional further planning reading: Dana Nau, Malik Ghallab, and Paolo Traverso. *Automated Planning: Theory and Practice*. Morgan Kaufmann. This book goes beyond what is required here, but is useful for you if you want a deeper planning treatment.